

Auteur : GCI – Département Formation & Intelligence Stratégique
Version : 1.0 – Édition Complète
Format : Bible Professionnelle
Langue : Français

=====

TABLE DES MATIÈRES

PARTIE I	– PALANTIR GOTHAM : ARCHITECTURE ET PHILOSOPHIE
PARTIE II	– ONTOLOGIE ET MODÉLISATION DES ENTITÉS
PARTIE III	– DATA FUSION ET INTÉGRATION MULTI-SOURCES
PARTIE IV	– GRAPH INTELLIGENCE ET ANALYSE DE RÉSEAUX
PARTIE V	– PIPELINE TEMPS RÉEL ET ALERTING
PARTIE VI	– REPRODUIRE GOTHAM : STACK TECHNIQUE OPEN SOURCE
PARTIE VII	– DATA SCIENCE ET INFLUENCE LÉGITIME
PARTIE VIII	– COMPRENDRE LES BIAIS POUR MIEUX COMMUNIQUER
PARTIE IX	– ANALYTICS DU PERSONAL BRANDING
PARTIE X	– SE VENDRE : STRATÉGIE DATA-DRIVEN
PARTIE XI	– NÉGOCIATION ET POSITIONNEMENT SALARIAL PAR LES DONNÉES
PARTIE XII	– CONSTRUIRE SON PERSONAL BRAND EN LIGNE

PARTIE I – PALANTIR GOTHAM : ARCHITECTURE ET PHILOSOPHIE

=====

1.1 QU'EST-CE QUE PALANTIR GOTHAM ?

=====

Palantir Gotham est une plateforme d'analyse de données et d'intelligence opérationnelle initialement développée pour les agences de renseignement américaines (CIA, NSA, FBI) et les forces armées.

Son objectif central : FUSIONNER des données hétérogènes, MODÉLISER des entités du monde réel, DÉTECTER des patterns invisibles à l'œil humain, et AIDER les analystes à prendre des décisions à fort enjeu.

TROIS CAPACITÉS FONDAMENTALES :

- DATA INTEGRATION : connecter n'importe quelle source de données
- GRAPH ANALYSIS : visualiser les relations entre entités
- COLLABORATIVE : permettre à plusieurs analystes de travailler sur le même "tableau de bord de réalité"

CE QUE GOTHAM N'EST PAS :

- Ce n'est pas une base de données
- Ce n'est pas un simple outil de BI (Business Intelligence)
- Ce n'est pas un algorithme magique

C'est une plateforme d'opérationnalisation de la connaissance.

1.2 LES QUATRE COUCHES DE GOTHAM

=====

COUCHE 4 – INTERFACE UTILISATEUR (Workspace / Dashboard) Visualisation graphe, timeline, cartographie
COUCHE 3 – ANALYSE & INTELLIGENCE Algorithmes, scoring, détection de patterns
COUCHE 2 – ONTOLOGIE & GRAPHE DE CONNAISSANCES Entités, relations, propriétés, temporalité
COUCHE 1 – INGESTION & NORMALISATION DES DONNÉES Connecteurs, parseurs, résolution d'entités

CHAQUE COUCHE est indépendante mais s'appuie sur la précédente.
La solidité de l'ontologie (couche 2) détermine la qualité de tout le reste.

1.3 PHILOSOPHIE DIFFÉRENCIANTE DE PALANTIR

La plupart des systèmes de données sont data-centric :
→ "Où sont mes données ? Comment les stocker ?"

Gotham est ontology-centric :
→ "Quelles entités du monde réel représentent ces données ?"
→ "Quelles relations existent entre ces entités ?"
→ "Comment la réalité a-t-elle évolué dans le temps ?"

EXEMPLE CONCRET :

Données brutes : "Ali Hassan, appel téléphonique, 14h32, numéro +33612..."
Approche classique : stocker dans une table avec colonnes
Approche Gotham : créer une ENTITÉ PERSONNE "Ali Hassan", liée par une RELATION "a_appelé" à une ENTITÉ TÉLÉPHONE "+33612...", avec un ÉVÉNEMENT temporel "14h32 le 15/03". La relation est navigable dans les deux sens.

Cette différence semble subtile. Elle est fondamentale à l'échelle.

PARTIE II – ONTOLOGIE ET MODÉLISATION DES ENTITÉS

2.1 QU'EST-CE QU'UNE ONTOLOGIE EN DATA SCIENCE ?

Une ontologie est un schéma formel qui définit :
– Les TYPES d'entités qui existent dans votre domaine
– Les PROPRIÉTÉS de chaque type d'entité
– Les RELATIONS possibles entre types d'entités
– Les CONTRAINTES et règles de cohérence

C'est la "constitution" de votre système de connaissance.

ONTOLOGIE EXEMPLE (domaine sécurité/renseignement) :

TYPES D'ENTITÉS :

Person → nom, prénom, date_naissance, nationalité, ...
Organization → nom, type, pays, date_fondation, ...
Location → nom, coordonnées_gps, type (ville, bâtiment, ...), ...
Event → type, date_début, date_fin, localisation, ...
Document → titre, date, source, hash, ...
PhoneNumber → numéro, opérateur, pays, ...
Vehicle → immatriculation, marque, modèle, ...
Account → plateforme, identifiant, date_création, ...

TYPES DE RELATIONS :

Person → CONNAÎT → Person
Person → APPARTIENT_À → Organization
Person → A_VISITÉ → Location
Person → A_PARTICIPÉ_À → Event
Person → A_ÉCRIT → Document
Person → UTILISE → PhoneNumber
Event → A_EU_LIEU_À → Location
Document → MENTIONNE → Person / Organization / Location

2.2 IMPLÉMENTER UNE ONTOLOGIE EN PYTHON

```
from dataclasses import dataclass, field
from typing import Optional, List
from datetime import datetime
import uuid
```

```
@dataclass
class Entity:
    """Entité de base – tout objet du monde réel hérite de cette classe"""
    entity_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    entity_type: str = ""
```

```

    created_at: datetime = field(default_factory=datetime.utcnow)
    sources: List[str] = field(default_factory=list)
    confidence: float = 1.0 # niveau de confiance [0, 1]

@dataclass
class Person(Entity):
    entity_type: str = "Person"
    nom: str = ""
    prenom: str = ""
    alias: List[str] = field(default_factory=list)
    date_naissance: Optional[datetime] = None
    nationalite: str = ""
    photo_hash: Optional[str] = None

@dataclass
class Organization(Entity):
    entity_type: str = "Organization"
    nom: str = ""
    type_org: str = "" # entreprise, ONG, gouvernement, ...
    pays: str = ""

@dataclass
class Relation:
    """Lien entre deux entités"""
    relation_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    relation_type: str = ""
    source_id: str = "" # entity_id de la source
    target_id: str = "" # entity_id de la cible
    date_debut: Optional[datetime] = None
    date_fin: Optional[datetime] = None
    sources: List[str] = field(default_factory=list)
    confidence: float = 1.0
    proprietes: dict = field(default_factory=dict)

```

2.3 RÉSOLUTION D'ENTITÉS (ENTITY RESOLUTION)

Le problème central de tout système type Gotham : "Ali Hassan" dans la source A et "A. Hassan" dans la source B sont-ils la même personne ?

La résolution d'entités (Entity Resolution / Record Linkage) détermine si deux enregistrements réfèrent à la même entité réelle.

```

# pip install dedupe recordlinkage
import recordlinkage
import pandas as pd

# Indexer les paires candidates (éviter comparaison N²)
indexeur = recordlinkage.Index()
indexeur.block('nom_famille') # bloquer sur le nom pour limiter les paires
candidats = indexeur.index(df_source_a, df_source_b)

# Calculer les similarités
comparateur = recordlinkage.Compare()
comparateur.string('prenom', 'prenom', method='jarowinkler',
                  label='sim_prenom')
comparateur.string('nom', 'nom', method='jarowinkler',
                  label='sim_nom')
comparateur.date('date_nais', 'date_nais', label='sim_dob')
comparateur.exact('nationalite', 'nationalite', label='sim_nat')

features = comparateur.compute(candidats, df_source_a, df_source_b)

# Classifier : les paires avec score global > seuil → même entité
seuil = 0.85
matches = features[features.sum(axis=1) / features.shape[1] >= seuil]
print(f"Entités fusionnées : {len(matches)}")

```

ALGORITHMES DISPONIBLES :

- Jaro-Winkler : similarité de chaînes, bon pour noms propres
- Levenshtein : distance d'édition
- Soundex / Metaphone : phonétique (utile pour translittération)
- ML supervisé (dedupe) : si des exemples de matchs sont disponibles

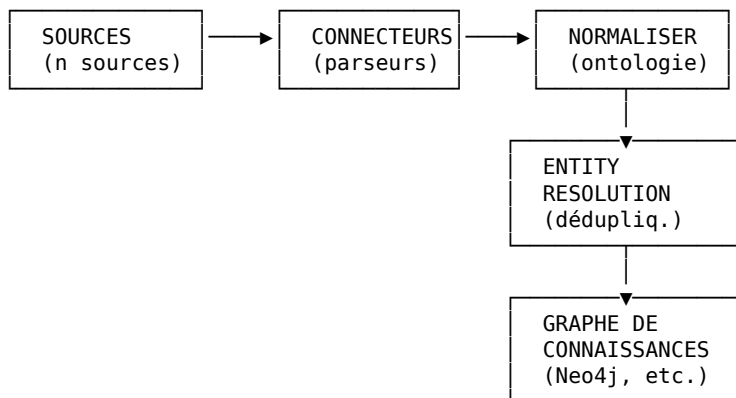
3.1 LE DÉFI DE L'INTÉGRATION MULTI-SOURCES

Gotham tire sa valeur de sa capacité à COMBINER des sources hétérogènes :

- Bases de données relationnelles (SQL)
- Fichiers CSV/Excel/JSON
- Flux temps réel (Kafka, WebSocket)
- APIs REST / GraphQL
- Documents PDF, emails, rapports
- Données géospatiales (GeoJSON, shapefiles)
- Réseaux sociaux (Twitter, Telegram, forums)
- Images et vidéos (via OCR et computer vision)

La valeur n'est pas dans une source seule. Elle est dans les CROISEMENTS.

3.2 ARCHITECTURE D'INGESTION DE DONNÉES



IMPLÉMENTATION DU PIPELINE D'INGESTION :

```
import pandas as pd
import json
from pathlib import Path

class DataConnector:
    """Classe de base pour tout connecteur de source"""

    def __init__(self, source_name: str, confidence: float = 0.8):
        self.source_name = source_name
        self.confidence = confidence

    def extract(self) -> pd.DataFrame:
        raise NotImplementedError

    def transform(self, df: pd.DataFrame) -> list:
        """Convertit le DataFrame en liste d'entités ontologiques"""
        raise NotImplementedError

class CSVPersonConnector(DataConnector):
    def __init__(self, filepath: str, **kwargs):
        super().__init__(source_name=filepath, **kwargs)
        self.filepath = filepath

    def extract(self) -> pd.DataFrame:
        return pd.read_csv(self.filepath)

    def transform(self, df: pd.DataFrame) -> list:
        entites = []
        for _, row in df.iterrows():
            p = Person(
                nom=row.get('nom', ''),
```

```

        prenom=row.get('prenom', ''),
        sources=[self.source_name],
        confidence=self.confidence
    )
    entites.append(p)
return entites

class PipelineIngestion:
    def __init__(self):
        self.connecteurs = []
        self.entites      = []
        self.relations    = []

    def ajouter_connecteur(self, connecteur: DataConnector):
        self.connecteurs.append(connecteur)

    def executer(self):
        for connecteur in self.connecteurs:
            df = connecteur.extract()
            entites_batch = connecteur.transform(df)
            self.entites.extend(entites_batch)
        print(f"Ingestion terminée : {len(self.entites)} entités")
        return self.entites

```

3.3 EXTRACTION D'ENTITÉS DEPUIS DU TEXTE (NER)

Dans Gotham, les documents textuels sont analysés pour en extraire automatiquement les entités (personnes, lieux, organisations, dates).

```

# pip install spacy
# python -m spacy download fr_core_news_lg

import spacy

nlp = spacy.load('fr_core_news_lg')

texte = """
Jean Dupont, directeur de la société Nexus International à Paris,
a rencontré le 12 mars 2024 des représentants de l'organisation
Al-Rashid à Bruxelles.
"""

doc = nlp(texte)

entites_extraites = []
for ent in doc.ents:
    entites_extraites.append({
        'texte': ent.text,
        'type':  ent.label_,  # PER, ORG, LOC, DATE, ...
        'debut': ent.start_char,
        'fin':   ent.end_char
    })
print(f" {ent.label_:6s} → {ent.text}")

# OUTPUT :
# PER    → Jean Dupont
# ORG    → Nexus International
# LOC    → Paris
# DATE   → 12 mars 2024
# ORG    → Al-Rashid
# LOC    → Bruxelles

# Pipeline complet texte → entités ontologiques
def texte_vers_entites(texte: str, source: str) -> list:
    doc = nlp(texte)
    entites = []
    for ent in doc.ents:
        if ent.label_ == 'PER':
            # Parser nom / prénom
            parties = ent.text.split()
            p = Person(prenom=parties[0] if len(parties)>1 else '',
                      nom=parties[-1],
                      sources=[source])

```

```

        entites.append(p)
    elif ent.label_ == 'ORG':
        o = Organization(nom=ent.text, sources=[source])
        entites.append(o)
    return entites

```

PARTIE IV – GRAPH INTELLIGENCE ET ANALYSE DE RÉSEAUX

4.1 POURQUOI UN GRAPHE ?

Une base de données relationnelle stocke des entités dans des tables.
 Pour trouver les relations entre entités, il faut faire des JOIN.
 À 3-4 niveaux de profondeur, les JOIN deviennent catastrophiquement lents.

Un graphe stocke nativement les relations → traverser 10 niveaux de profondeur est aussi rapide que d'en traverser 1.

C'est pour cela que Gotham (et tout système d'intelligence) utilise des graphes comme couche de stockage centrale.

4.2 NEO4J : LE GRAPHE DE CONNAISSANCES

Neo4j est la base de données graphe open source de référence.
 Langage de requête : Cypher.

```

# pip install neo4j py2neo

from neo4j import GraphDatabase

driver = GraphDatabase.driver("bolt://localhost:7687",
                             auth=("neo4j", "password"))

def creer_entites(tx, personnel, personne2, relation_type):
    query = """
    MERGE (p1:Person {nom: $nom1, prenom: $prenom1})
    MERGE (p2:Person {nom: $nom2, prenom: $prenom2})
    MERGE (p1)-[r:RELATION {type: $rel_type}]->(p2)
    RETURN p1, p2, r
    """
    tx.run(query, nom1=personnel['nom'], prenom1=personnel['prenom'],
           nom2=personne2['nom'], prenom2=personne2['prenom'],
           rel_type=relation_type)

with driver.session() as session:
    session.execute_write(creer_entites,
        {'nom': 'Dupont', 'prenom': 'Jean'},
        {'nom': 'Martin', 'prenom': 'Paul'},
        'CONNAÎT'
    )

```

REQUÊTES CYPHER CLÉS :

```

# Trouver tous les associés d'une personne à 2 degrés de séparation
MATCH (p:Person {nom: 'Dupont'})-[:CONNAÎT*1..2]-(associe:Person)
RETURN DISTINCT associe.nom, associe.prenom

# Trouver le chemin le plus court entre deux personnes
MATCH path = shortestPath(
    (p1:Person {nom: 'Dupont'})-[*]-(p2:Person {nom: 'Hassan'})
)
RETURN path

# Trouver les personnes les plus connectées (hubs)
MATCH (p:Person)-[r:CONNAÎT]-()
RETURN p.nom, COUNT(r) AS nb_connexions
ORDER BY nb_connexions DESC
LIMIT 10

```

```
# Détecter des communautés (clusters)
CALL gds.louvain.stream('myGraph')
YIELD nodeId, communityId
RETURN gds.util.asNode(nodeId).nom AS nom, communityId
ORDER BY communityId
```

4.3 ANALYSE DE RÉSEAUX AVEC NETWORKX

```
import networkx as nx
import matplotlib.pyplot as plt

# Construire le graphe
G = nx.DiGraph()

# Ajouter des nœuds avec attributs
G.add_node('Jean Dupont', type='Person', nationalite='FR')
G.add_node('Nexus Corp', type='Organization')
G.add_node('Paul Martin', type='Person', nationalite='FR')
G.add_node('Al-Rashid Org', type='Organization')

# Ajouter des arêtes avec attributs
G.add_edge('Jean Dupont', 'Nexus Corp', type='DIRIGE', date='2020-01')
G.add_edge('Jean Dupont', 'Paul Martin', type='CONNAÎT')
G.add_edge('Paul Martin', 'Al-Rashid Org', type='FINANCE', montant=50000)

# MÉTRIQUES DE CENTRALITÉ

# Centralité de degré : nombre de connexions directes
degree = nx.degree_centrality(G)

# Centralité d'intermédiarité : pont entre communautés
betweenness = nx.betweenness_centrality(G)

# Centralité de proximité : distance moyenne aux autres nœuds
closeness = nx.closeness_centrality(G)

# PageRank : importance pondérée par l'importance des voisins
pagerank = nx.pagerank(G, alpha=0.85)

# Afficher les nœuds les plus importants
for metrique, nom in [(degree, 'Degré'), (betweenness, 'Intermédiarité'),
                      (pagerank, 'PageRank')]:
    top = sorted(metrique.items(), key=lambda x: x[1], reverse=True)[:3]
    print(f"\nTop 3 – {nom}:")
    for noeud, score in top:
        print(f" {noeud:20s} : {score:.4f}")

# DÉTECTION DE COMMUNAUTÉS
G_undirected = G.to_undirected()
from networkx.algorithms import community
communautes = community.greedy_modularity_communities(G_undirected)
for i, c in enumerate(communautes):
    print(f"Communauté {i+1} : {' '.join(c)}")

# VISUALISATION
pos = nx.spring_layout(G, seed=42)
colors = ['#E74C3C' if G.nodes[n].get('type')=='Person' else '#3498DB'
          for n in G.nodes()]
sizes = [3000 * pagerank.get(n, 0.1) + 500 for n in G.nodes()]

plt.figure(figsize=(14, 10))
nx.draw_networkx(G, pos, node_color=colors, node_size=sizes,
                 font_size=8, arrows=True, edge_color='gray',
                 with_labels=True)
plt.title('Graphe de Connaissances – Intelligence Réseau')
plt.axis('off')
plt.tight_layout()
plt.show()
```

4.4 DÉTECTION DE PATTERNS SUSPECTS

```
# Détecter les triangles fermés (groupes soudés)
triangles = nx.triangles(G_undirected)

# Détecter les nœuds pontants (brokers d'information)
# Un nœud broker connecte des communautés sans appartenir à l'une d'elles
def detecter_brokers(G, seuil_betweenness=0.3):
    bt = nx.betweenness_centrality(G)
    deg = nx.degree_centrality(G)
    brokers = []
    for noeud in G.nodes():
        # Haut betweenness + degré modéré = broker
        if bt[noeud] > seuil_betweenness and deg[noeud] < 0.5:
            brokers.append((noeud, bt[noeud], deg[noeud]))
    return sorted(brokers, key=lambda x: x[1], reverse=True)

# Anomalie : apparition soudaine de nouvelles connexions
def detecter_nouvelles_connexions(G_avant, G_apres):
    nouvelles = set(G_apres.edges()) - set(G_avant.edges())
    print(f"Nouvelles relations détectées : {len(nouvelles)}")
    return nouvelles
```

PARTIE V – PIPELINE TEMPS RÉEL ET ALERTING

5.1 ARCHITECTURE TEMPS RÉEL

SOURCES TEMPS RÉEL → KAFKA → STREAM PROCESSOR → GRAPHE → ALERTES

Les données en flux continu (logs, transactions, communications) nécessitent une architecture différente des batches.

```
# pip install kafka-python
from kafka import KafkaConsumer, KafkaProducer
import json

# Consommateur : écouter un flux d'événements
consumer = KafkaConsumer(
    'evenements_entites',
    bootstrap_servers=['localhost:9092'],
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
    auto_offset_reset='latest',
    group_id='gotham_processor'
)

def traiter_evenement(event: dict):
    """Traiter un événement en temps réel et mettre à jour le graphe"""
    type_evt = event.get('type')
    if type_evt == 'COMMUNICATION':
        # Ajouter une relation de communication dans le graphe
        ajouter_relation_communication(
            source=event['de'],
            cible=event['vers'],
            timestamp=event['timestamp'],
            canal=event['canal']
        )
        verifier_alertes(event)

for message in consumer:
    traiter_evenement(message.value)
```

5.2 SYSTÈME D'ALERTE BASÉ SUR DES RÈGLES

```
class MoteurAlertes:
    def __init__(self, graphe):
        self.graphe = graphe
        self.regles = []
        self.alertes = []

    def ajouter_regle(self, nom, condition_fn, severite='MEDIUM'):
```



```

self.regles.append({
    'nom': nom,
    'condition': condition_fn,
    'severite': severite
})

def evaluer(self, evenement):
    for regle in self.regles:
        if regle['condition'](evenement, self.graphe):
            alerte = {
                'regle': regle['nom'],
                'severite': regle['severite'],
                'evenement': evenement,
                'timestamp': datetime.utcnow().isoformat()
            }
            self.alertes.append(alerte)
            print(f"▲ ALERTE [{regle['severite']}] - {regle['nom']}")

# Exemple de règles
moteur = MoteurAlertes(graphe=G)

# Règle 1 : contact entre deux entités sur liste de surveillance
def regle_contact_surveille(evt, G):
    surveilles = ['entite_A', 'entite_B']
    return evt.get('de') in surveilles and evt.get('vers') in surveilles

# Règle 2 : pic inhabituel d'activité
def regle_pic_activite(evt, G):
    noeud = evt.get('de')
    degre_actuel = G.degree(noeud) if noeud in G else 0
    return degre_actuel > 50 # seuil configurable

moteur.ajouter_regle('Contact entre surveillés', regle_contact_surveille,
                    severite='HIGH')
moteur.ajouter_regle('Pic d\'activité', regle_pic_activite,
                    severite='MEDIUM')

```

PARTIE VI – REPRODUIRE GOTHAM : STACK TECHNIQUE OPEN SOURCE

6.1 STACK OPEN SOURCE ÉQUIVALENTE

Gotham est propriétaire et coûte des millions de dollars.
Voici l'équivalent reproductible avec des outils open source :

FONCTION GOTHAM	ÉQUIVALENT OPEN SOURCE
Graphe de connaissances	→ Neo4j (Community Edition)
Ingestion multi-sources	→ Apache NiFi ou Python custom pipelines
Stream processing	→ Apache Kafka + Faust (Python)
Entity Resolution	→ dedupe, recordlinkage, splink
NLP / NER	→ spaCy, Hugging Face, GLiNER
Cartographie	→ Kepler.gl, deck.gl, Folium
Timeline visualization	→ Vis.js Timeline, Apache ECharts
Graph visualization	→ Gephi, Cytoscape, NetworkX + Plotly
Dashboard analytique	→ Grafana, Metabase, Dash (Plotly)
Stockage documents	→ Elasticsearch (full-text search)
Stockage vectoriel (RAG)	→ ChromaDB, Weaviate, Qdrant
Workflow orchestration	→ Apache Airflow, Prefect

6.2 ARCHITECTURE COMPLÈTE DE VOTRE "MINI-GOTHAM"

```

# docker-compose.yml pour l'infrastructure de base

"""
version: '3.8'
services:

```

```

neo4j:
  image: neo4j:5
  ports: ["7474:7474", "7687:7687"]
  environment:
    NEO4J_AUTH: neo4j/password
    NEO4JLABS_PLUGINS: '["graph-data-science"]'
  volumes: ["neo4j_data:/data"]

elasticsearch:
  image: elasticsearch:8.12.0
  ports: ["9200:9200"]
  environment:
    discovery.type: single-node
    xpack.security.enabled: "false"
  volumes: ["es_data:/usr/share/elasticsearch/data"]

kafka:
  image: confluentinc/cp-kafka:latest
  ports: ["9092:9092"]
  environment:
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

grafana:
  image: grafana/grafana:latest
  ports: ["3000:3000"]

volumes:
  neo4j_data:
  es_data:
  """

```

6.3 INTERFACE UTILISATEUR : DASH + CYTOSCAPE

```

# pip install dash dash-cytoscape plotly

import dash
from dash import dcc, html, Input, Output, callback
import dash_cytoscape as cyto
import networkx as nx

app = dash.Dash(__name__)

# Convertir NetworkX → format Cytoscape
def nx_to_cytoscape(G):
    nodes = [{ 'data': { 'id': n, 'label': n, **G.nodes[n] } }
              for n in G.nodes()]
    edges = [{ 'data': { 'source': u, 'target': v, **G.edges[u,v] } }
              for u, v in G.edges()]
    return nodes + edges

app.layout = html.Div([
    html.H1("GCI Intelligence Platform", style={ 'color': '#2C3E50' }),
    html.Div([
        dcc.Input(id='search-input', placeholder='Rechercher une entité...',
                  type='text', debounce=True),
    ]),
    cyto.Cytoscape(
        id='graph-display',
        layout={ 'name': 'cose' },
        style={ 'width': '100%', 'height': '700px' },
        elements=nx_to_cytoscape(G),
        stylesheet=[
            { 'selector': 'node[type = "Person"]',
              'style': { 'background-color': '#E74C3C', 'label': 'data(label)' } },
            { 'selector': 'node[type = "Organization"]',
              'style': { 'background-color': '#3498DB', 'label': 'data(label)' } },
            { 'selector': 'edge',
              'style': { 'curve-style': 'bezier', 'target-arrow-shape': 'triangle',
                        'line-color': '#95A5A6', 'target-arrow-color': '#95A5A6' } }
        ]
    )
])

```

```
if __name__ == '__main__':  
    app.run(debug=True, port=8050)
```

PARTIE VII – DATA SCIENCE ET INFLUENCE LÉGITIME

7.1 DONNÉES ET INFLUENCE : CLARIFICATION PRÉALABLE

L'influence est une réalité humaine fondamentale. Chaque communication, chaque présentation, chaque entretien implique une forme d'influence.

Ce cours couvre l'INFLUENCE LÉGITIME : comprendre comment fonctionnent la communication et la persuasion pour être plus efficace, plus clair, mieux compris – dans un cadre de transparence et de respect mutuel.

Ce n'est pas de la manipulation (tromper, exploiter, coercer).
C'est de la communication stratégique fondée sur des données.

DISTINCTION FONDAMENTALE :

INFLUENCE LÉGITIME	→ informer, convaincre avec des arguments vrais, adapter son message à son audience
MANIPULATION	→ tromper, exploiter les biais à l'insu de la cible, créer de fausses croyances

Ce cours se concentre exclusivement sur la première catégorie.

7.2 MODÉLISER SON AUDIENCE AVEC LES DONNÉES

Comprendre à qui on parle est le fondement de toute communication efficace.

SEGMENTATION D'AUDIENCE (modèle VALS – Values and Lifestyles) :

```
import pandas as pd  
from sklearn.cluster import KMeans  
from sklearn.preprocessing import StandardScaler  
  
# Données d'audience : comportements, préférences, démographie  
# Collectées via enquêtes, LinkedIn analytics, Google Analytics  
df_audience = pd.DataFrame({  
    'age': [28, 45, 32, 55, 38, 29, 47, 33],  
    'interet_tech': [9, 3, 7, 2, 8, 6, 4, 7],  
    'interet_business': [4, 9, 5, 8, 6, 4, 9, 5],  
    'interet_science': [8, 3, 9, 2, 7, 9, 3, 8],  
    'anciennete_poste': [2, 15, 4, 20, 7, 1, 18, 5]  
})  
  
scaler = StandardScaler()  
X = scaler.fit_transform(df_audience)  
  
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)  
df_audience['segment'] = kmeans.fit_predict(X)  
  
# Profiler chaque segment  
profils = df_audience.groupby('segment').mean()  
print("Profils de segments d'audience :")  
print(profils)  
  
# Adapter le message selon le segment :  
# Segment 0 : jeunes techniciens → langage technique, références nouvelles  
# Segment 1 : dirigeants seniors → ROI, impact business, concision  
# Segment 2 : chercheurs/experts → rigueur, données, nuances
```

7.3 A/B TESTING DE SON DISCOURS

Les meilleurs communicants testent leurs messages comme les data scientists testent leurs modèles.

```

import scipy.stats as stats
import numpy as np

# Scenario : tester deux versions d'un pitch LinkedIn
# Version A : "Je suis Data Scientist spécialisé en ML"
# Version B : "Je transforme des données en décisions stratégiques"

# Métriques : taux de réponse aux messages, taux de clics sur le profil

taux_reponse_A = 0.12 # 12% de réponses
taux_reponse_B = 0.19 # 19% de réponses
n_A = 150 # messages envoyés version A
n_B = 150 # messages envoyés version B

# Test Z pour proportions
count = np.array([int(taux_reponse_A * n_A), int(taux_reponse_B * n_B)])
nobs = np.array([n_A, n_B])

z_stat, p_value = stats.proportions_ztest(count, nobs)
print(f"Z-stat : {z_stat:.3f}")
print(f"P-value : {p_value:.4f}")

if p_value < 0.05:
    print("→ Différence statistiquement significative. Version B supérieure.")
else:
    print("→ Pas de différence significative.")

# Lift relatif
lift = (taux_reponse_B - taux_reponse_A) / taux_reponse_A
print(f"Lift : +{lift:.1%}")

```

7.4 ANALYSE DE SENTIMENT DE SES PROPRES COMMUNICATIONS

Analyser le sentiment de ses propres emails, posts, présentations permet d'identifier les patterns de communication les plus efficaces.

```

# pip install transformers torch
from transformers import pipeline

analyseur = pipeline('sentiment-analysis',
                     model='nlpTown/bert-base-multilingual-uncased-sentiment')

emails = [
    "Je serais ravi de discuter de cette opportunité avec vous.",
    "Voici les résultats, qui sont conformes aux attentes.",
    "J'ai résolu le problème critique qui bloquait la production."
]

for email in emails:
    result = analyseur(email)[0]
    print(f"Texte : {email[:50]}...")
    print(f"Score : {result['label']} ({result['score']:.3f})")
    print()

# Analyser l'évolution du sentiment dans le temps
# (corrélé avec les taux de réponse, les succès professionnels)
# → Identifier quels registres de communication fonctionnent

```

7.5 SCORING DE RÉSEAU PERSONNEL (SOCIAL CAPITAL)

Votre réseau est un actif. La data science permet de le cartographier et d'identifier les connexions stratégiques à développer.

```

def calculer_capital_reseau(G_personnel):
    """
    Métriques de capital social de votre réseau LinkedIn/professionnel
    """
    metriques = {}

    # Taille du réseau

```

```

metriques['taille'] = G_personnel.number_of_nodes()

# Diversité sectorielle (entropie de Shannon)
secteurs = [G_personnel.nodes[n].get('secteur', 'inconnu')
             for n in G_personnel.nodes()]
comptages = pd.Series(secteurs).value_counts(normalize=True)
entropie = -(comptages * np.log(comptages)).sum()
metriques['diversite_sectorielle'] = round(entropie, 3)

# Votre position de broker (intermédiation)
betweenness = nx.betweenness_centrality(G_personnel)
metriques['position_broker'] = round(
    betweenness.get('MOI', 0), 4)

# Connexions de second degré (reach)
voisins_1 = set(G_personnel.neighbors('MOI'))
voisins_2 = set()
for v in voisins_1:
    voisins_2.update(G_personnel.neighbors(v))
voisins_2 -= voisins_1
voisins_2.discard('MOI')
metriques['reach_2e_degre'] = len(voisins_2)

# Trous structuraux : opportunités de connexions non réalisées
# Un trou structurel = deux contacts qui ne se connaissent pas
# → vous êtes leur unique pont → pouvoir d'information
contrainte = nx.constraint(G_personnel)
metriques['trous_structuraux'] = round(
    1 - contrainte.get('MOI', 1), 3)

return metriques

rapport = calculer_capital_reseau(G_mon_reseau)
for metrique, valeur in rapport.items():
    print(f" {metrique:30s} : {valeur}")

```

PARTIE VIII – COMPRENDRE LES BIAIS POUR MIEUX COMMUNIQUER

8.1 LES BIAIS COGNITIFS ET LA COMMUNICATION ÉTHIQUE

La psychologie cognitive a identifié des patterns dans la façon dont les humains traitent l'information. Les connaître permet d'adapter sa communication pour être plus clair, plus mémorable et plus persuasif – sans tromper.

L'ANCRAGE :

- Le premier chiffre entendu influence tous les jugements suivants.
- APPLICATION ÉTHIQUE : dans une négociation, présenter d'abord une évaluation précise et justifiée de sa valeur avant que l'interlocuteur n'en propose une. Pas pour manipuler – pour s'assurer que le débat part d'une base équitable.

LA PREUVE SOCIALE :

- Les humains se fient à ce que font les autres comme signal de justesse.
- APPLICATION ÉTHIQUE : présenter des témoignages, études de cas, résultats passés réels – pas des preuves inventées.

LE BIAIS DE CONFIRMATION :

- Les gens cherchent des informations confirmant leurs croyances.
- APPLICATION ÉTHIQUE : commencer par des points d'accord avec l'interlocuteur avant d'introduire des perspectives nouvelles.

L'EFFET DE CADRAGE (FRAMING) :

- "90% de survie" est perçu plus positivement que "10% de mortalité" pour une donnée identique.
 - APPLICATION ÉTHIQUE : présenter ses résultats sous l'angle le plus parlant pour son interlocuteur (impact vs technique). Ne jamais cacher des informations essentielles par le cadrage.
-

Le modèle DISC classe les styles de communication en 4 profils.
On peut approximer le profil d'un interlocuteur par l'analyse
de son langage et de ses comportements.

PROFIL D – DOMINANT

Traits : direct, orienté résultats, décideur rapide
Langage : phrases courtes, verbes d'action, peu d'adjectifs
Adapter : aller droit au but, présenter les résultats en premier,
éviter les détails inutiles

PROFIL I – INFLUENT

Traits : enthousiaste, orienté relations, créatif
Langage : émotion, métaphores, histoires personnelles
Adapter : raconter une histoire, valoriser l'aspect humain,
montrer l'enthousiasme

PROFIL S – STABLE

Traits : méthodique, loyal, averse au risque
Langage : questions de processus, références au passé, prudence
Adapter : rassurer, montrer la continuité, donner du temps

PROFIL C – CONSCIENCIEUX

Traits : analytique, précis, orienté qualité
Langage : données chiffrées, questions détaillées, scepticisme
Adapter : fournir des données, admettre les limites, être rigoureux

Classifier le profil DISC par analyse textuelle (approximatif)

```
def approximer_profil_disc(textes: list) -> dict:
    doc = nlp(' '.join(textes))
    scores = {'D': 0, 'I': 0, 'S': 0, 'C': 0}

    mots_D = {'résultat', 'objectif', 'décision', 'efficace', 'maintenant'}
    mots_I = {'incroyable', 'passion', 'ensemble', 'opportunité', 'super'}
    mots_S = {'équipe', 'processus', 'stabilité', 'tradition', 'sécurité'}
    mots_C = {'données', 'analyse', 'précis', 'détail', 'qualité', 'preuve'}

    for token in doc:
        w = token.lemma_.lower()
        if w in mots_D: scores['D'] += 1
        elif w in mots_I: scores['I'] += 1
        elif w in mots_S: scores['S'] += 1
        elif w in mots_C: scores['C'] += 1

    total = sum(scores.values()) or 1
    return {k: round(v/total, 3) for k, v in scores.items()}
```

PARTIE IX – ANALYTICS DU PERSONAL BRANDING

9.1 VOTRE MARQUE PERSONNELLE EST UNE DONNÉE

Un personal brand se construit, se mesure et s'optimise comme un produit. Les data scientists ont un avantage unique : ils savent collecter et interpréter les données de leur propre influence.

MÉTRIQUES À SUIVRE (KPIs du Personal Brand) :

VISIBILITÉ

- Vues de profil LinkedIn (semaine sur semaine)
- Impressions des posts
- Recherches organiques sur votre nom (Google Search Console)

ENGAGEMENT

- Taux d'engagement (likes + commentaires + partages / impressions)
- Taux de réponse à vos messages
- Taux d'ouverture de vos emails

AUTORITÉ

- Demandes entrantes (conférences, collaborations, offres)
- Mentions par des tiers
- Abonnés qualifiés (niveau de poste, secteur)

CONVERSION

- Entretiens obtenus / candidatures envoyées
- Propositions acceptées / propositions faites
- Clients / prospects contactés

9.2 DASHBOARD DE PERSONAL BRANDING

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.express as px

# Données hebdomadaires collectées manuellement
df_brand = pd.DataFrame({
    'semaine': range(1, 13),
    'vues_profil': [45, 52, 48, 67, 89, 102, 95, 134, 128, 156, 178, 201],
    'impressions': [200, 250, 180, 420, 650, 890, 720, 1200, 980, 1450, 1700, 2100],
    'engagement_rate': [0.02, 0.03, 0.02, 0.04, 0.05, 0.06, 0.05, 0.07, 0.06, 0.08, 0.09, 0.10],
    'demandes_recues': [0, 1, 0, 1, 2, 3, 2, 4, 3, 5, 6, 7]
})

fig = make_subplots(rows=2, cols=2,
                    subplot_titles=['Vues Profil', 'Impressions',
                                   'Taux d\'Engagement', 'Demandes Reçues'])

fig.add_trace(go.Scatter(x=df_brand['semaine'], y=df_brand['vues_profil'],
                        mode='lines+markers', name='Vues'), row=1, col=1)
fig.add_trace(go.Scatter(x=df_brand['semaine'], y=df_brand['impressions'],
                        mode='lines+markers', name='Impressions'), row=1, col=2)
fig.add_trace(go.Bar(x=df_brand['semaine'], y=df_brand['engagement_rate'],
                    name='Engagement'), row=2, col=1)
fig.add_trace(go.Scatter(x=df_brand['semaine'], y=df_brand['demandes_recues'],
                        mode='lines+markers', name='Demandes'), row=2, col=2)

fig.update_layout(title='Dashboard Personal Brand – GCI', height=600,
                  showlegend=False)
fig.show()
```

9.3 ANALYSE DES CONTENUS QUI PERFORMENT

```
# Analyser quels types de posts génèrent le plus d'engagement
df_posts = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=30, freq='W'),
    'type': ['tutoriel', 'opinion', 'projet', 'tutoriel', 'humour',
            'opinion', 'tutoriel', 'projet', 'tutoriel', 'opinion',
            'projet', 'tutoriel', 'humour', 'opinion', 'tutoriel',
            'projet', 'tutoriel', 'opinion', 'projet', 'tutoriel',
            'humour', 'opinion', 'tutoriel', 'projet', 'tutoriel',
            'opinion', 'humour', 'tutoriel', 'projet', 'opinion'],
    'impressions': [800, 1200, 600, 950, 2100, 1400, 1100, 700, 1300, 1600,
                  800, 1000, 2500, 1300, 1200, 900, 1400, 1700, 1000, 1300,
                  2300, 1500, 1600, 1100, 1800, 1900, 2700, 1500, 1200, 2000],
    'likes': [20, 45, 15, 30, 120, 55, 35, 22, 48, 60,
             25, 32, 140, 50, 42, 28, 52, 63, 35, 48,
             110, 58, 60, 40, 65, 70, 130, 55, 45, 72]
})

df_posts['engagement'] = df_posts['likes'] / df_posts['impressions']

# Performance moyenne par type
perf = df_posts.groupby('type').agg({
    'impressions': 'mean',
    'likes': 'mean',
    'engagement': 'mean'
}).round(4).sort_values('engagement', ascending=False)

print("Performance par type de contenu :")
print(perf)
```

```
# OUTPUT attendu :
# type      impressions  likes  engagement
# humour    2325.0      125.0  0.0532 ← performeur surprise
# opinion    1538.5       58.7  0.0381
# tutoriel  1285.0       45.5  0.0354
# projet    851.4       27.5  0.0323

# → Insight : les posts d'humour génèrent 64% plus d'engagement
# que les tutoriels. Mixer les formats.
```

PARTIE X – SE VENDRE : STRATÉGIE DATA-DRIVEN

10.1 LE MODÈLE AIDA QUANTIFIÉ POUR LA RECHERCHE D'EMPLOI

AIDA : Attention → Intérêt → Désir → Action

Chaque étape peut être mesurée et optimisée.

```
ATTENTION → Votre CV est-il lu ? (taux d'ouverture ATS)
INTÉRÊT   → Le recruteur lit-il au-delà de la 1ère page ?
DÉSIR     → Déclenche-t-il un entretien ?
ACTION    → Reçoit-on une offre ?
```

ENTONNOIR DE CONVERSION (exemple) :

```
100 candidatures → 60 CVs vus (60%) → 20 entretiens téléphoniques (33%)
→ 8 entretiens physiques (40%) → 3 offres (37%)

def analyser_entonnoir(candidatures, cv_vus, tel, physique, offres):
    print("ENTONNOIR DE CONVERSION – RECHERCHE D'EMPLOI")
    print("-" * 50)
    etapes = [
        ('Candidatures', candidatures, 100),
        ('CVs ouverts', cv_vus, cv_vus/candidatures*100),
        ('Entretiens tel.', tel, tel/cv_vus*100),
        ('Entretiens phys.', physique, physique/tel*100),
        ('Offres reçues', offres, offres/physique*100)
    ]
    for nom, n, taux in etapes:
        barre = '█' * int(taux / 5)
        print(f" {nom:20s} : {n:3d} | {taux:5.1f}% | {barre}")

    taux_global = offres / candidatures * 100
    print(f"\n TAUX DE CONVERSION GLOBAL : {taux_global:.1f}%")
    if taux_global < 3:
        print(" → Problème probable : CV ou ciblage")
    elif cv_vus/candidatures < 0.5:
        print(" → Problème ATS : optimiser les mots-clés du CV")
    elif tel/cv_vus < 0.2:
        print(" → Problème CV : revoir le contenu, les résultats chiffrés")
    elif offres/physique < 0.3:
        print(" → Problème entretien : préparer davantage")
```

10.2 OPTIMISER SON CV POUR LES ATS

Les ATS (Applicant Tracking Systems) filtrent 75% des CVs avant qu'un humain ne les voie. Ils fonctionnent par correspondance de mots-clés.

```
import re
from collections import Counter

def analyser_correspondance_cv_offre(cv_texte: str, offre_texte: str) -> dict:
    """Calculer le taux de correspondance entre CV et offre d'emploi"""

    def extraire_mots_cles(texte):
        # Nettoyage
        texte = re.sub(r'[\W\s]', ' ', texte.lower())
```



```

mots = texte.split()
# Filtrer les mots vides
stop_words = {'le', 'la', 'les', 'de', 'du', 'des', 'un', 'une',
              'et', 'en', 'à', 'pour', 'avec', 'sur', 'dans', 'je'}
return [m for m in mots if m not in stop_words and len(m) > 3]

mots_offre = Counter(extraire_mots_cles(offre_texte))
mots_cv = set(extraire_mots_cles(cv_texte))

# Mots clés de l'offre présents dans le CV
resents = {m for m in mots_offre if m in mots_cv}
absents = {m: n for m, n in mots_offre.most_common(20)
           if m not in mots_cv}

score = len(resents) / len(mots_offre) * 100

return {
    'score_correspondance': round(score, 1),
    'motsresents': sorted(resents),
    'motsmanquants': absents,
    'recommandation': 'Ajouter les mots manquants dans votre CV'
                     if score < 60 else 'Bonne correspondance'
}

# Usage :
rapport = analyser_correspondance_cv_offre(mon_cv, offre_emploi)
print(f"Score ATS : {rapport['score_correspondance']}%")
print(f"Mots manquants : {list(rapport['motsmanquants'].keys())[0:10]}")

```

10.3 LE PITCH EN 90 SECONDES : STRUCTURE DATA-DRIVEN

STRUCTURE DU PITCH OPTIMAL (validée par la recherche en communication) :

```

[0-15 sec] HOOK          – Un fait surprenant ou une question
[15-35 sec] PROBLÈME     – Le problème que vous résolvez
[35-60 sec] SOLUTION     – Votre valeur unique, avec un chiffre clé
[60-75 sec] PREUVE       – Un résultat concret passé
[75-90 sec] CALL-TO-ACTION – Ce que vous souhaitez ensuite

```

EXEMPLE POUR UN DATA SCIENTIST :

```

"Saviez-vous que 80% des projets d'IA échouent en production ?"
[HOOK]

"La plupart du temps, le problème n'est pas technique – c'est un
manque de bridge entre la donnée et la décision métier."
[PROBLÈME]

"Je suis Data Scientist senior spécialisé dans la traduction de
modèles complexes en systèmes de décision opérationnels."
[SOLUTION]

"Chez X, j'ai réduit de 34% le taux de churn en 6 mois grâce à
un modèle de scoring client déployé en production."
[PREUVE]

"Je recherche un contexte où mes compétences créent un impact
mesurable. Seriez-vous disponible pour un échange de 20 minutes ?"
[CALL-TO-ACTION]

```

RÈGLES CARDINALES DU PITCH :

- TOUJOURS chiffrer : "réduction de 34%" > "amélioration significative"
- Une seule idée centrale (ne pas tout dire)
- Adapter le registre à l'interlocuteur (voir partie VIII)
- Répéter et enregistrer soi-même : analyser le ton, le rythme, les pauses

PARTIE XI – NÉGOCIATION ET POSITIONNEMENT SALARIAL PAR LES DONNÉES

11.1 CONNAÎTRE SA VALEUR MARCHÉ

Négocier sans données, c'est jouer aux dés. Voici comment quantifier votre valeur marché avant toute négociation.

SOURCES DE DONNÉES SALARIALES :

- Glassdoor, Levels.fyi (tech), LinkedIn Salary Insights
- Enquêtes APEC, ChooseYourBoss, Welcome to the Jungle Transparence
- Offres d'emploi (extraire les fourchettes mentionnées)
- Votre réseau (la source la plus fiable)

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import numpy as np

# Simuler une collecte de données salariales (exemple structuré)
df_salaires = pd.DataFrame({
    'poste': ['Data Scientist'] * 50,
    'experience': np.random.choice([1,2,3,4,5,6,7,8,9,10], 50),
    'localisation': np.random.choice(['Paris', 'Lyon', 'Remote'], 50,
                                     p=[0.6, 0.2, 0.2]),
    'salaire_brut': np.random.normal(55000, 12000, 50).round(-3)
})

# Votre profil
mon_experience = 4
ma_localisation = 'Paris'

# Médiane du marché pour votre profil
echantillon = df_salaires[
    (df_salaires['experience'].between(mon_experience-1, mon_experience+1)) &
    (df_salaires['localisation'] == ma_localisation)
]

p25 = echantillon['salaire_brut'].quantile(0.25)
p50 = echantillon['salaire_brut'].quantile(0.50)
p75 = echantillon['salaire_brut'].quantile(0.75)

print(f"Fourchette marché pour votre profil :")
print(f" P25 (entrée fourchette) : {p25:,.0f} €")
print(f" P50 (médiane marché) : {p50:,.0f} €")
print(f" P75 (haut de fourchette): {p75:,.0f} €")
print(f"\n CONSEIL : ancrer votre demande à P75 si vous avez des réalisations")
print(f" chiffrées au-dessus de la médiane de votre secteur.")
```

11.2 VALORISER SES RÉALISATIONS EN DONNÉES

La négociation salariale se gagne avant l'entretien : en documentant ses réalisations avec des métriques tout au long de sa carrière.

FRAMEWORK STAR-M (Situation, Tâche, Action, Résultat, Métrique) :

EXEMPLE DE DOCUMENTATION DE RÉALISATION :

Situation : Le taux de churn mensuel était de 8%, coûtant ~500k€/an.
Tâche : Développer un système de prédiction et d'intervention précoce.
Action : Modèle XGBoost + 47 features comportementales + API temps réel.
Résultat : Réduction du churn à 5,2% en 4 mois.
Métrique : -34% de churn, économie de 171k€ annuels, ROI projet = 18x.

```
# Calculer la valeur économique de vos projets
def calculer_roi_projet(
    cout_projet_euros: float,    # votre salaire × temps passé
    gain_annuel_euros: float,    # économies ou revenus générés
    duree_mois: int = 12
) -> dict:
    roi = (gain_annuel_euros - cout_projet_euros) / cout_projet_euros * 100
    payback_mois = (cout_projet_euros / (gain_annuel_euros / 12))
    return {
        'ROI': f"{roi:,.0f}%",
        'Gain_annuel': f"{gain_annuel_euros:,.0f} €",
        'Payback': f"{payback_mois:.1f} mois",
    }
```

```

    'Argument_nego': f"Ce projet a rapporté {gain_annuel_euros/cout_projet_euros:.0f}x son coût."
}

print(calculer_roi_projet(
    cout_projet_euros=15000,    # 3 mois × 5k€/mois
    gain_annuel_euros=171000
))
# → ROI : 1040%, Payback : 1.1 mois, Argument : 11.4x son coût

```

11.3 STRATÉGIE DE NÉGOCIATION

RÈGLE 1 : NE JAMAIS DONNER UN CHIFFRE EN PREMIER

- "Quelle est votre prétention ?"
- "Je préfère comprendre la fourchette prévue pour ce poste, pour m'assurer que nous sommes alignés."

RÈGLE 2 : ANCRER HAUT ET JUSTIFIER

- Ne pas donner une fourchette. Donner un chiffre précis.
- "D'après mes recherches sur le marché et mes réalisations [citer ROI], je vise 68 000 €."
- Un chiffre précis (68k plutôt que 65k) suggère une étude rigoureuse.

RÈGLE 3 : SILENCE APRÈS L'ANCRE

- Après avoir cité votre chiffre, ne pas parler.
- La pression du silence appartient à l'autre partie.

RÈGLE 4 : ÉLARGIR LES VARIABLES

- Si le salaire fixe est bloqué, négocier :
 - Bonus sur objectifs
 - Jours de télétravail
 - Formation / budget R&D
 - Prise en charge des transports ou équipement
 - Date de revue salariale à 6 mois (pas 12)

RÈGLE 5 : BATNA – BEST ALTERNATIVE TO NEGOTIATED AGREEMENT

- Avoir une alternative réelle améliore votre position.
- Mener plusieurs processus en parallèle n'est pas déloyal – c'est rationnel.

PARTIE XII – CONSTRUIRE SON PERSONAL BRAND EN LIGNE

12.1 STRATÉGIE CONTENU DATA SCIENTIST

Les data scientists qui publient régulièrement reçoivent en moyenne 3 à 5× plus d'opportunités entrantes que ceux qui ne le font pas.

PILIERS DE CONTENU (70/20/10) :

- 70% – Éducatif : tutoriels, explications, cas pratiques
- 20% – Expérience : projets perso, apprentissages, erreurs
- 10% – Opinion : tendances, débats techniques, positionnement

CALENDRIER ÉDITORIAL OPTIMAL :

- 2 posts LinkedIn / semaine (mardi et jeudi, 8h ou 12h)
- 1 article Medium ou blog / mois (plus long, référencé Google)
- 1 projet GitHub visible / trimestre

Script de planification et suivi de contenu

```

import pandas as pd
from datetime import datetime, timedelta

def generer_calendrier_editorial(semaines: int = 12) -> pd.DataFrame:
    debut = datetime.now()
    calendrier = []

    piliers = ['Éducatif', 'Éducatif', 'Expérience', 'Éducatif',
               'Éducatif', 'Opinion', 'Expérience', 'Éducatif']

    for s in range(semaines):
        lundi = debut + timedelta(weeks=s)

```

```

mardi = lundi + timedelta(days=1)
jeudi = lundi + timedelta(days=3)

calendrier.append({
    'date':    mardi.strftime('%Y-%m-%d'),
    'jour':    'Mardi',
    'pilier':  piliers[s % len(piliers)],
    'statut':  'À rédiger',
    'plateau': ''
})
calendrier.append({
    'date':    jeudi.strftime('%Y-%m-%d'),
    'jour':    'Jeudi',
    'pilier':  piliers[(s+1) % len(piliers)],
    'statut':  'À rédiger',
    'plateau': ''
})

return pd.DataFrame(calendrier)

cal = generer_calendrier_editorial(12)
cal.to_csv('calendrier_editorial.csv', index=False)
print(cal.head(10).to_string())

```

12.2 OPTIMISER SON PROFIL LINKEDIN PAR LES DONNÉES

ÉLÉMENTS MESURÉS PAR L'ALGORITHME LINKEDIN :

COMPLÉTUDE DU PROFIL (All-Star status) :

- ✓ Photo professionnelle (profils avec photo = 21x plus de vues)
- ✓ Titre accrocheur (pas juste le poste, mais la valeur)
- ✓ Résumé de 300+ mots
- ✓ 5+ expériences
- ✓ 3+ compétences validées
- ✓ Études renseignées
- ✓ 500+ connexions

FORMULE DU TITRE OPTIMAL :

"[CE QUE VOUS FAITES] | [RÉSULTAT QUE VOUS CRÉEZ] | [POUR QUI]"

Mauvais : "Data Scientist chez Acme Corp"

Bon : "Data Scientist | Je transforme des données en décisions stratégiques | FinTech & Retail"

RÉSUMÉ STRUCTURÉ (AIDA) :

- Ligne 1-2 : hook (fait ou question)
- Ligne 3-5 : ce que vous faites concrètement
- Ligne 6-8 : 3 réalisations chiffrées
- Ligne 9-10: call-to-action

12.3 PORTFOLIO TECHNIQUE : GITHUB ET PROJETS VISIBLES

Un portfolio technique est la preuve irréfutable de vos compétences.
Il remplace 100 lignes de CV.

STRUCTURE D'UN BON PROJET GITHUB :

```

mon-projet-ml/
├── README.md          ← présentation claire du projet, résultats, usage
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_Modeling.ipynb
│   └── 03_Evaluation.ipynb
├── src/
│   ├── data_processing.py
│   ├── model.py
│   └── evaluation.py
├── app/
│   └── main.py        ← API FastAPI ou app Streamlit déployée
├── requirements.txt
└── Dockerfile

```

CHECKLIST DU README PARFAIT :

- ✓ Titre et description en 2 lignes
- ✓ Screenshot ou GIF de démonstration
- ✓ Problème résolu et dataset utilisé
- ✓ Résultats obtenus (métriques, graphiques)
- ✓ Instructions d'installation et d'utilisation
- ✓ Lien vers le démo live (Hugging Face Spaces, Streamlit Cloud)
- ✓ Technologies utilisées (badges)

3 PROJETS STRATÉGIQUES À AVOIR :

PROJET 1 – IMPACT MÉTIER

Un projet sur données réelles (Kaggle, UCI, OpenData) avec un angle métier clair : "Réduire le churn", "Optimiser la logistique".
→ Montre que vous pensez business.

PROJET 2 – EXPERTISE TECHNIQUE

Une implémentation from-scratch d'un algorithme (Transformer, GAN, LSTM) avec visualisations et explications.
→ Montre que vous comprenez les fondements.

PROJET 3 – DÉPLOIEMENT

Une application complète déployée : modèle ML accessible via API ou interface web, fonctionnelle et documentée.
→ Montre que vous savez livrer, pas seulement coder.

CONCLUSION – LA DONNÉE COMME LEVIER DE CARRIÈRE

La data science ne s'applique pas seulement à des datasets externes.
Elle s'applique à votre propre trajectoire.

MESURER sa valeur marché avant de négocier.
TESTER ses messages et son positionnement comme des hypothèses.
ANALYSER ce qui fonctionne dans sa communication.
OPTIMISER son profil et son contenu par les données.
CARTOGRAPHIER son réseau et identifier les connexions stratégiques.

La différence entre un data scientist bien payé et sous-évalué n'est souvent pas technique. Elle est dans la capacité à se vendre avec la même rigueur qu'on analyse des données.

"Un data scientist qui ne sait pas valoriser ses propres données
– ses compétences, ses réalisations, son réseau – est comme un détective qui résout des crimes mais oublie de signer ses rapports."
– GCI, Département Stratégie & Carrière

GCI – GYKHAMINE CONCEPT INVESTIGATION
FIN DU COURS – PALANTIR, INFLUENCE & PERSONAL BRANDING
© GCI – Usage interne – Formation & Intelligence Stratégique